

Homework 1

Cesar Cabrera Garcia

COP 4045-042 Python Programming

Fall 2026

Problem 1 - Quadratic Equations

Source file: Homework1/p1_CabreraGarcia_Cesar.py

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Solve and graph quadratic equations entered from the terminal

import math
import matplotlib.pyplot as pyplot

# -----
# Initial variable declarations
# -----
number_of_points = 150
domain_padding = 2.0
default_domain_radius = 5.0
a_text = ""
a_value = 0.0
b_value = 0.0
c_value = 0.0
discriminant = 0.0
x_one = 0.0
x_two = 0.0
x_opt = 0.0
x_min = 0.0
x_max = 0.0
dx = 0.0
x_value = 0.0
y_value = 0.0
point_counter = 0
xs = []
ys = []

while True:
    a_text = input("Enter a: ")

    if a_text == "":
        break

    a_value = float(a_text)
    b_value = float(input("Enter b: "))
    c_value = float(input("Enter c: "))

    discriminant = b_value**2 - 4 * a_value * c_value
    x_opt = -b_value / (2 * a_value)

    # Pick the graph domain so the roots or vertex are visible.
    if discriminant < 0:
        print("no real solutions")
        x_min = x_opt - default_domain_radius
        x_max = x_opt + default_domain_radius
    elif discriminant == 0:
        x_one = -b_value / (2 * a_value)
        print("one solution:", x_one)
        x_min = x_one - default_domain_radius
        x_max = x_one + default_domain_radius
    else:
        x_one = (-b_value - math.sqrt(discriminant)) / (2 * a_value)
        x_two = (-b_value + math.sqrt(discriminant)) / (2 * a_value)
        print("two solutions: x1=", x_one, "x2=", x_two)

    if x_one < x_two:
        x_min = x_one - domain_padding
        x_max = x_two + domain_padding
    else:
        x_min = x_two - domain_padding
        x_max = x_one + domain_padding
```

```

xs = []
ys = []
dx = (x_max - x_min) / (number_of_points - 1)
x_value = x_min
point_counter = 0

while point_counter < number_of_points:
    y_value = a_value * x_value**2 + b_value * x_value + c_value
    xs.append(x_value)
    ys.append(y_value)
    x_value = x_value + dx
    point_counter = point_counter + 1

pyplot.plot(xs, ys, "ro-")
pyplot.xlabel("x")
pyplot.ylabel("y")
pyplot.title("Quadratic Function")
pyplot.grid(True)
pyplot.show()
pyplot.clf()

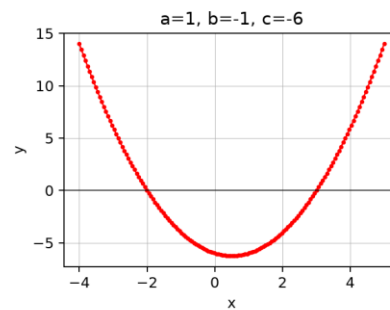
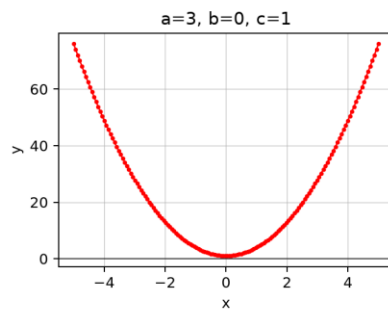
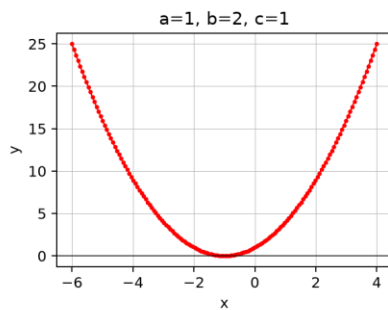
```

Output Evidence

Problem 1 Terminal Output

Enter a: Enter b: Enter c: one solution: -1.0
 Enter a: Enter b: Enter c: no real solutions
 Enter a: Enter b: Enter c: two solutions: x1= -2.0 x2= 3.0
 Enter a:

Problem 1 Quadratic Charts - 150 Points Each



Problem 2 - Pythagorean Numbers

Source file: Homework1/p2_CabreraGarcia_Cesar.py

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Find all Pythagorean triples up to a user-entered bound

# -----
# Initial variable declarations
# -----
n_value = 0
a_value = 0
b_value = 0
c_value = 0
triples = []
triple_index = 0

def find_Pythagorean(n):
    found_triples = []

    # Try every ordered value so both (3, 4, 5) and (4, 3, 5) are included.
    for a_value in range(1, n + 1):
        for b_value in range(1, n + 1):
            for c_value in range(1, n + 1):
                if a_value**2 + b_value**2 == c_value**2:
                    found_triples.append((a_value, b_value, c_value))

    return found_triples

n_value = int(input("Enter n: "))
triples = find_Pythagorean(n_value)

triple_index = 0

while triple_index < len(triples):
    print(triples[triple_index])
    triple_index = triple_index + 1

if len(triples) == 0:
    print("No triples found.")
```

Output Evidence

Problem 2 Terminal Output

```
Enter n: (3, 4, 5)
(4, 3, 5)
```

Problem 3 - Duplicated Substrings

Source file: Homework1/p3_CabreraGarcia_Cesar.py

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Locate duplicated substrings in user-entered text

# -----
# Initial variable declarations
# -----
s = ""
n_value = 0
duplicated_substring = ""
maximum_duplicated_substring = ""

def find_dup_str(s, n):
    text_length = 0
    start_one = 0
    start_two = 0
    first_piece = ""
    second_piece = ""

    for current_character in s:
        text_length = text_length + 1

    if n <= 0:
        return ""

    # Start the second substring after the first one ends so they do not overlap.
    while start_one <= text_length - n:
        first_piece = s[start_one:start_one + n]
        start_two = start_one + n

        while start_two <= text_length - n:
            second_piece = s[start_two:start_two + n]

            if first_piece == second_piece:
                return first_piece

            start_two = start_two + 1

        start_one = start_one + 1

    return ""

def find_max_dup(s):
    text_length = 0
    current_length = 0
    current_result = ""

    for current_character in s:
        text_length = text_length + 1

    current_length = text_length - 1

    # Search from longest to shortest so the first match is the answer.
    while current_length > 0:
        current_result = find_dup_str(s, current_length)

        if current_result != "":
            return current_result
```

```
        current_length = current_length - 1

    return ""

s = input("Enter string: ")
n_value = int(input("Enter substring length: "))

duplicated_substring = find_dup_str(s, n_value)
print("Duplicated substring:", duplicated_substring)

maximum_duplicated_substring = find_max_dup(s)
print("Longest duplicated substring:", maximum_duplicated_substring)
```

Output Evidence

Problem 3 Terminal Output

```
Enter string: Enter substring length: Duplicated substring: bcd
Longest duplicated substring: bcd
```

Problem 4 - Function Visualization

Source file: Homework1/p4_CabreraGarcia_Cesar.py

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Display a table and chart for a user-entered math function

import math
import matplotlib.pyplot as pyplot

# -----
# Initial variable declarations
# -----
fun_str = ""
xmin = 0.0
xmax = 0.0
domain = (xmin, xmax)
ns = 0

def plot_function(fun_str, domain, ns):
    xs = []
    ys = []
    xmin = domain[0]
    xmax = domain[1]
    sample_counter = 0
    dx = 0.0
    x = xmin
    y = 0.0

    if ns == 1:
        dx = 0.0
    else:
        dx = (xmax - xmin) / (ns - 1)

    # Evaluate the typed expression for each x value.
    while sample_counter < ns:
        xs.append(x)
        y = eval(fun_str)
        ys.append(y)
        x = x + dx
        sample_counter = sample_counter + 1

    print("{:>10} {:>10}".format("x", "y"))
    print("-----")

    sample_counter = 0
    while sample_counter < ns:
        print("{:+10.4f} {:+10.4f}".format(xs[sample_counter], ys[sample_counter]))
        sample_counter = sample_counter + 1

    pyplot.plot(xs, ys, "ro-")
    pyplot.xlabel("x")
    pyplot.ylabel("y")
    pyplot.title(fun_str)
    pyplot.grid(True)
    pyplot.show()
    pyplot.clf()

fun_str = input("Enter function with variable x: ")
ns = int(input("Enter number of samples: "))
xmin = float(input("Enter xmin: "))
xmax = float(input("Enter xmax: "))
domain = (xmin, xmax)
```

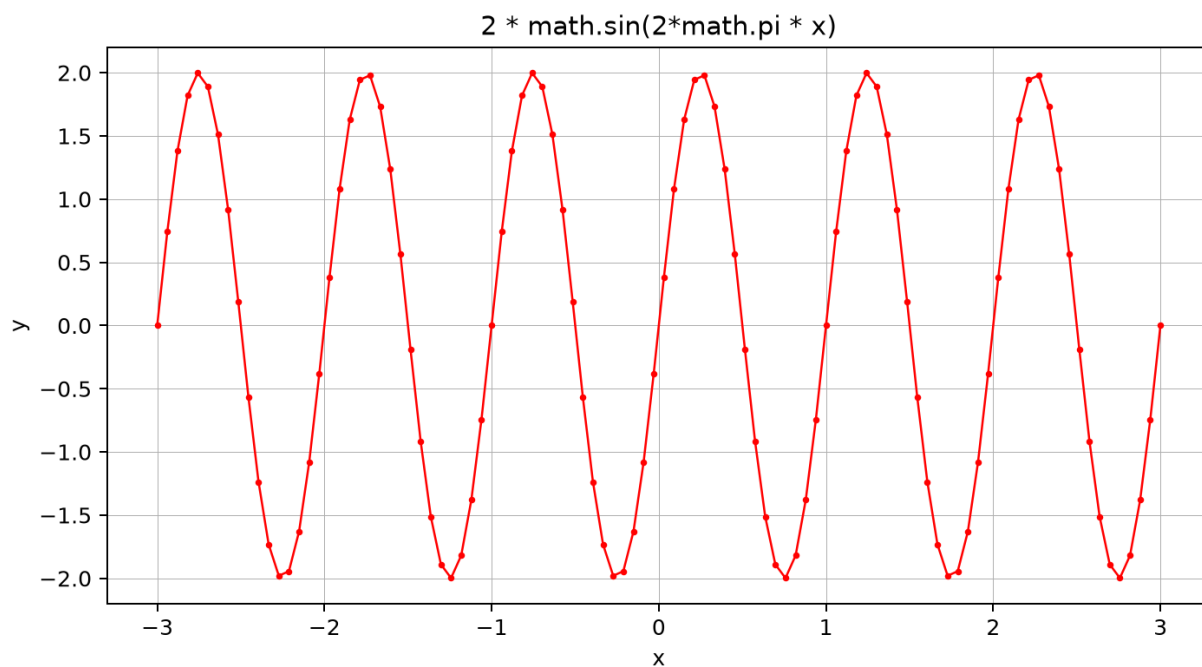
```
plot_function(fun_str, domain, ns)
```

Output Evidence

Problem 4 Terminal Output

Enter function with variable x: Enter number of samples: Enter xmin: Enter xmax: x y

```
-----  
-1.0000 +0.0000  
-0.5000 -0.0000  
+0.0000 +0.0000  
+0.5000 +0.0000  
+1.0000 -0.0000
```



Problem 5 - Interactive Caesar Cipher

Source file: Homework1/p5/p5_CabreraGarcia_Cesar.py

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Interactive Caesar cipher tool with letter frequency reporting

import string

ALPHABET_LENGTH = 26
LOWERCASE_ALPHABET = string.ascii_lowercase
UPPERCASE_ALPHABET = string.ascii_uppercase

def caesar_cipher(text, shift):
    encrypted_text = ""
    normalized_shift = shift % ALPHABET_LENGTH

    for current_character in text:
        if current_character in LOWERCASE_ALPHABET:
            original_index = LOWERCASE_ALPHABET.index(current_character)
            shifted_index = (original_index + normalized_shift) % ALPHABET_LENGTH
            encrypted_text = encrypted_text + LOWERCASE_ALPHABET[shifted_index]
        elif current_character in UPPERCASE_ALPHABET:
            original_index = UPPERCASE_ALPHABET.index(current_character)
            shifted_index = (original_index + normalized_shift) % ALPHABET_LENGTH
            encrypted_text = encrypted_text + UPPERCASE_ALPHABET[shifted_index]
        else:
            encrypted_text = encrypted_text + current_character

    return encrypted_text

def caesar_decipher(cyphertext, shift):
    return caesar_cipher(cyphertext, -shift)

def letter_frequency(text):
    frequency_report = ""

    for current_letter in LOWERCASE_ALPHABET:
        letter_count = 0

        for current_character in text:
            if current_character.lower() == current_letter:
                letter_count = letter_count + 1

        frequency_report = frequency_report + current_letter + ": " + str(letter_count)

    if current_letter != "z":
        frequency_report = frequency_report + "\n"

    return frequency_report

def main():
    print("Caesar Cipher Tool")

    while True:
        print("")
        print("1. Encrypt, analyze, and decrypt a message")
        print("2. Quit")
```

```
menu_choice = input("Enter menu option: ")

if menu_choice == "1":
    plain_text = input("Enter message: ")
    shift_value = int(input("Enter shift value: "))

    cipher_text = caesar_cipher(plain_text, shift_value)
    frequency_report = letter_frequency(cipher_text)
    deciphered_text = caesar_decipher(cipher_text, shift_value)

    print("")
    print("Ciphered text:")
    print(cipher_text)
    print("")
    print("Letter frequency breakdown:")
    print(frequency_report)
    print("")
    print("Deciphered text:")
    print(deciphered_text)
elif menu_choice == "2":
    print("Program finished.")
    break
else:
    print("Invalid menu option. Please enter 1 or 2.")

if __name__ == "__main__":
    main()
```

Output Evidence

Problem 5 Terminal Output

Caesar Cipher Tool

```
1. Encrypt, analyze, and decrypt a message
2. Quit
Enter menu option: Enter message: Enter shift value:
Ciphered text:
Fdvk Xexqwx
```

Letter frequency breakdown:

```
a: 0
b: 0
c: 0
d: 1
e: 1
f: 1
g: 0
h: 0
i: 0
j: 0
k: 1
l: 0
m: 0
n: 0
o: 0
p: 0
q: 1
r: 0
s: 0
t: 0
u: 0
v: 1
w: 1
x: 3
y: 0
z: 0
```

```
Deciphered text:
Cash Ubuntu
```

```
1. Encrypt, analyze, and decrypt a message
2. Quit
Enter menu option: Program finished.
```

Problem 5 unittest Output

Coding-Agent Screenshots

Problem 5 coding-agent screenshot 1: Screenshot 2026-09-01 221053.png

Problem 5 help me write a python program for a homework assignment on a caesar cipher. i need three functions exactly called `caesar_cipher(text, shift)`, `caesar_decipher(cyphertext, shift)`, and `letter_frequency(text)`. i also need a main menu loop so the user can enter 1 to encrypt, analyze, and decrypt a message, or enter 2 to quit. pls make sure the cipher keeps capital letters, spaces and punctuation normal when it shifts. finally i need a completely separate python file to run unittests on the code

Worked for 18s >

I made both files and verified the test suite: **all 8 unit tests pass.**

Use these as your starting files, then rename `Yourlastname_Yourfirstname` to your actual name exactly as the assignment requires.

- [Download the Caesar cipher program](#)
- [Download the separate unittest file](#)

The program includes the exact required functions: `caesar_cipher(text, shift)`, `caesar_decipher(cyphertext, shift)`, and `letter_frequency(text)`. It preserves uppercase/lowercase, spaces, numbers, and punctuation; wraps around from `z` to `a`; supports negative and large shifts; and has the required `1 / 2` main menu.

One assignment-specific note: your instructions also require screenshots of **your prompts to the coding agent and the answers you receive**, so keep screenshots of this conversation for the Homework DOC/PDF.

Problem 5 Test File - unittest

Source file: Homework1/p5/p5_CabreraGarcia_Cesar_test.py

```
# Author Cesar Cabrera Garcia
# Course COP 4045-042 Python Programming
# Term Fall 2026
# Description Unit tests for the interactive Caesar cipher tool

import unittest

import p5_CabreraGarcia_Cesar

class TestCaesarCipher(unittest.TestCase):
    def test_caesar_cipher_preserves_case_and_spaces(self):
        cipher_text = p5_CabreraGarcia_Cesar.caesar_cipher("Cash Ubuntu", 3)
        self.assertEqual(cipher_text, "Fdvk Xexqwx")

    def test_caesar_cipher_wraps_alphabet(self):
        cipher_text = p5_CabreraGarcia_Cesar.caesar_cipher("Zorin OS", 2)
        self.assertEqual(cipher_text, "Bqtkp QU")

    def test_caesar_decipher_returns_clear_text(self):
        clear_text = p5_CabreraGarcia_Cesar.caesar_decipher("Njou", 1)
        self.assertEqual(clear_text, "Mint")

    def test_letter_frequency_ignores_case_and_non_letters(self):
        frequency_report = p5_CabreraGarcia_Cesar.letter_frequency("Cash! cash?")
        self.assertIn("a: 2", frequency_report)
        self.assertIn("c: 2", frequency_report)
        self.assertIn("h: 2", frequency_report)
        self.assertIn("s: 2", frequency_report)
        self.assertIn("b: 0", frequency_report)

if __name__ == "__main__":
    unittest.main()
```

